

Neural Multi-Objective Combinatorial Optimization for Flexible Job Shop Scheduling

26. 07. 16

박금빈



❖ Flexible Job Shop Scheduling Problem, FJSP

- 여러 작업(Job)의 공정(Operation)을 여러 기계(Machine)에 배정하고, 각 기계의 처리 순서를 결정하는 조합최적화 문제
- 각 작업은 정해진 순서의 공정으로 구성되며, 각 공정은 여러 호환 기계 중 하나에서 처리 가능
- 선택한 기계에 따라 처리시간이 달라지고, 각 기계는 동시에 하나의 공정만 처리 가능
- 따라서 기계 배정과 처리 순서를 동시에 결정해야 하며, 문제 크기가 커질수록 가능한 조합이 급격히 증가

❖ From Single-Objective to Multi-Objective FJSP

- 기존 FJSP 연구는 주로 전체 작업 완료시간인 Makespan 최소화에 집중
- 실제 제조 현장에서는 납기 지연, 작업 체류시간, 비용, 조기 완료에 따른 재고 부담 등을 함께 고려해야 함
- 본 논문은 Makespan, Total Tardiness, Total Earliness, Average Flowtime, Total Costs를 목적함수로 다룸

❖ Trade-off

- 빠른 기계 선택
 - Makespan·Tardiness 감소
 - Cost 증가
- 느린 기계 선택
 - Cost 감소
 - Makespan·Flowtime 증가

❖ Pareto Dominance

- 해 x_1 이 모든 목적에서 x_2 보다 나쁘지 않고, 적어도 하나의 목적에서 더 좋으면 x_1 이 x_2 를 지배함
- 다른 해에게 지배되는 스케줄은 실제 선택 후보에서 제외할 수 있음

❖ Pareto Set / Pareto Front

- Pareto optimal solution: 다른 어떤 해에도 지배되지 않는 스케줄
- Pareto Set: Pareto optimal한 실제 스케줄들의 집합
- Pareto Front: Pareto Set에 속한 스케줄들의 목적값 좌표 집합
- 여러 목적이 충돌하므로 정답 하나보다 서로 다른 절충관계를 가진 후보군을 제공하는 것이 중요

Schedule	Makespan	Cost	Result
A	8	100	Pareto
B	10	70	Pareto
C	14	40	Pareto
D	12	90	Dominated

$$X_1 \prec X_2$$

$f_i(x_1) \leq f_i(x_2)$ for all i , and strict inequality for at least one objective

* D는 B보다 느리고 비싸므로 제거

$$\lambda = (\lambda_1, \lambda_2, \dots, \lambda_M), \sum \lambda_i = 1$$

- 서로 다른 λ 입력
 - 서로 다른 목적 간 절충
 - 서로 다른 Pareto 후보 생성

❖ 기존 방식의 문제

- Weighted-sum 방식은 λ 하나당 해 하나로 여러 λ 를 반복하여 최적화해야함
- Metaheuristic은 큰 문제에서 실행시간이 증가

❖ 기존 Neural 방식의 문제

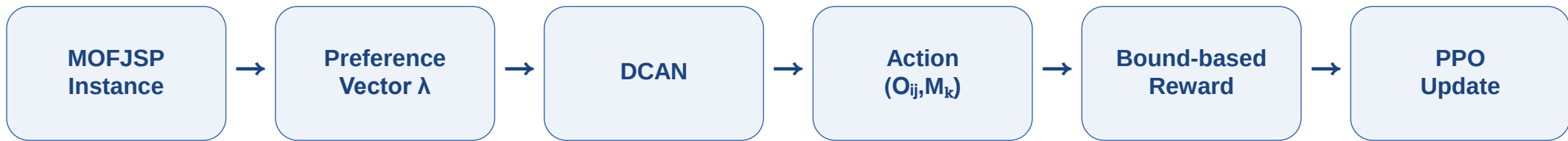
- Makespan 단일 목적 중심
- Dispatching rule만 선택
- 선호도마다 별도 Actor 필요
- 복잡한 동적 상태 표현 부족

❖ 본 논문이 해결하려고 하는 것

- 하나의 신경망
 - + 여러 preference
 - + 직접 Operation-Machine 선택
 - + 빠른 Pareto 해 생성

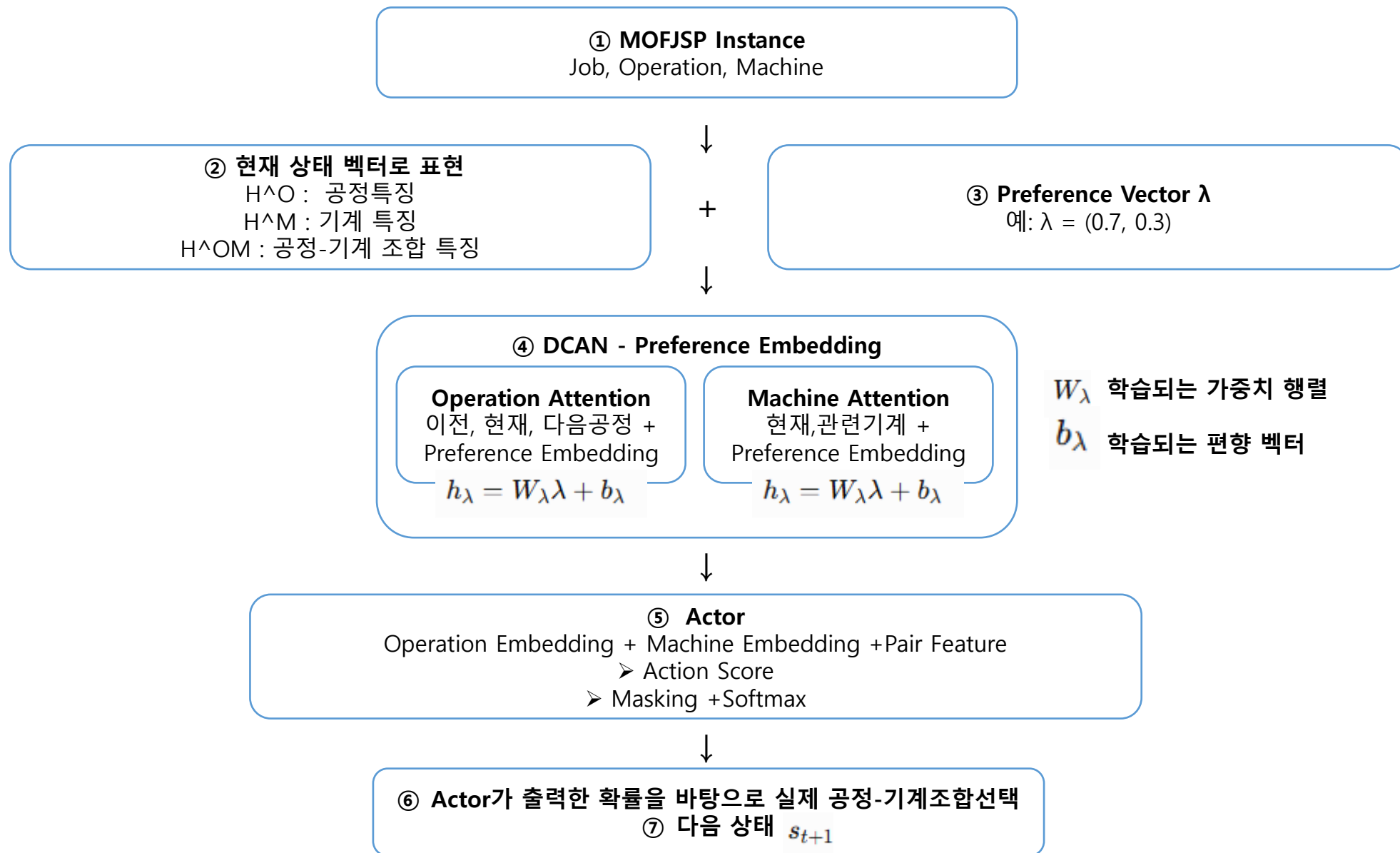
❖ Main Components

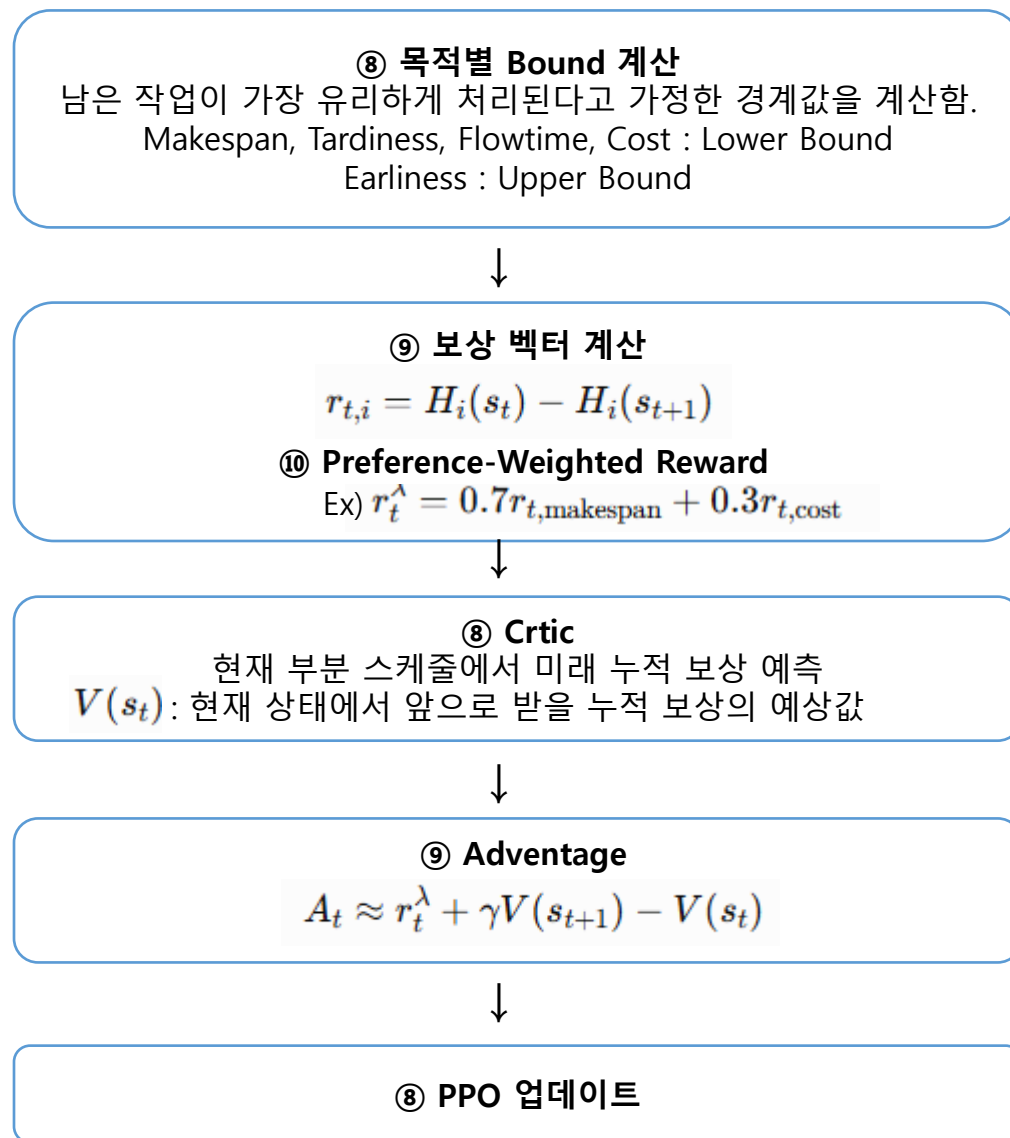
- Weighted-Sum Decomposition : 여러 선호도 벡터에 따라 다목적 문제를 여러 하위문제로 분해
- Dual Conditional Attention : Network (DCAN): 상태와 선호도를 함께 입력받아 행동 확률 계산
- Bound-Based Stepwise Reward : 공정 하나를 배정할 때마다 목적별 하한 또는 상한 변화로 보상 계산
- Decomposition-Based PPO: 목적별 보상을 선호도 가중합으로 결합해 하나의 정책을 학습



학습 후 하나의 고정된 DCAN에 여러 λ 를 입력하여 다양한 스케줄을 생성하고, 지배해를 제거해 Pareto Set을 구성

- 문제와 선호도 입력
 - DCAN이 행동 결정
 - 행동 후 보상 계산
 - PPO가 DCAN 학습
 - 학습 후 여러 선호도로 Pareto 해 생성





γ : 미래 보상 반영 정도를 정하는 하이퍼파라미터

계속 반복, Critic이 반복하면서 똑똑해짐

❖ Sequential Decision Process

$$s_t \rightarrow a_t \rightarrow r_t, s_{t+1}$$

- 공정을 하나씩 기계에 배정하며 부분 스케줄을 점진적으로 완성

❖ State

$$s_t = \{H^O, H^M, H^{OM}$$

}
공정, 기계, 공정-기계 pair 특징의 집합

- H^O : 공정 순서, 처리시간, 완료시간 하한, 남은 공정 등
- H^M : 기계 가용시간, 작업부하, 처리 가능한 공정 등
- H^{OM} : 특정 공정을 특정 기계에 배정할 때의 처리시간·비용 등

❖ Action Space

$$a_t = (O_{ij}, M_k) \in A(t)$$

현재 가능한 공정-기계 조합 하나를 선택

- 각 Job의 첫 번째 미배정 공정만 행동 후보가 됨
- 해당 공정을 처리할 수 있는 호환 기계와 조합해 $A(t)$ 를 구성
- 행동 후 공정 배정, 기계 가용시간, 다음 선택 가능 공정과 모든 상태 특징을 갱신

III Bound-Based Stepwise Reward

❖ Stepwise Reward

$$r_{t,i} = H_i(s_t) - H_i(s_{t+1})$$

행동 전후 목적 i 의 품질 측정값 변화

- 전체 스케줄 완성 후에만 보상을 주면 긴 의사결정에서 원인 추적이 어려움
- 중간 상태에서는 최종 목적값을 알 수 없으므로 lower bound 또는 upper bound를 사용
- 행동 후 품질 측정값이 커지면 음수 보상, 감소하면 양수 보상

❖ Completion-Time Lower Bound

$$C(O_{ij}, s_t) = C(O_{i(j-1)}, s_t) + \min_k p_{ij}^k$$

이전 공정의 최소 완료시간 + 가능한 기계 중 최소 처리시간

- 이미 배정된 공정은 실제 완료시간 사용
- 미배정 공정은 가장 유리한 기계를 가정한 낙관적 완료시간 사용
- Makespan, Tardiness, Flowtime, Cost는 하한 기반
- Earliness는 진행될수록 감소할 수 있으므로 상한 기반

❖ Preference-Weighted Reward

$$r_t^\lambda = \lambda^\top r_t = \sum_i \lambda_i r_{t,i}$$

- 목적별 보상을 현재 선호도에 맞게 하나의 PPO 보상으로 결합

III Decomposition-Based PPO

❖ Conditional Policy

$$\pi_{\theta}(a | s, \lambda)$$

현재 상태와 선호도가 주어졌을 때 행동을 선택할 확률

- 각 에피소드마다 새로운 MOFJSP 인스턴스와 preference vector를 샘플링
- 같은 문제에서도 서로 다른 λ 를 사용하여 다양한 하위문제를 학습
- 정책 확률에 따라 행동을 샘플링해 탐색을 유지
- 수집한 transition으로 GAE를 계산하고 clipped PPO로 정책을 업데이트
- 단계별 보상의 합이 최종 weighted-sum episodic reward와 일치하는 telescoping 구조
- Tchebycheff는 비선형·비가법적이라 단계 보상과 최종 목적의 정렬이 어려움
- NCO 선행연구에서 Tchebycheff가 weighted sum보다 항상 우수하지 않았음

❖ Training Objective

$$\max E[\sum_t \gamma^t \sum_i \lambda_i r_{t,i}]$$

다양한 문제와 선호도에서 기대 누적 보상을 최대화하는 하나의 정책 학습

❖ Algorithm

Algorithm 1 Decomposition-based PPO**Require:** Neural network with initialized parameters θ

```

1: Sample batch of  $n_B$  instances
2: for  $n_{ep} = 1, 2, \dots, N_{ep}$  do
3:   for  $b = 1, 2, \dots, n_B$  do
4:     Sample preferences  $\lambda \sim \Lambda$ 
5:     for  $t = 0, 1, \dots, |\mathcal{O}| - 1$  do
6:       Sample action  $a_{t,b} \sim \pi_\theta(s_{t,b}, \lambda)$ 
7:       Perform  $a_{t,b}$  and receive  $s_{t+1,b}^{det}$  and  $\mathbf{r}_{t,b}$ 
8:       Compute  $r_{t,b} = \lambda^\top \mathbf{r}_{t,b}$ 
9:       Collect transition  $(s_{t,b}, a_{t,b}, r_{t,b}, s_{t+1,b}, \lambda)$ 
10:    Compute generalized advantage estimates
11:  Compute PPO loss (Appendix C) and update  $\theta$ 
12:  if  $n_{ep} \bmod N_B = 0$  then Sample  $n_B$  new instances

```

▷ In Parallel

❖ WI-DAN vs. DCAN

Architecture	Preference Handling
WI-DAN	λ 를 공정·기계 특징 뒤에 단순 연결
DCAN	λ 를 Attention 내부의 가상 노드로 사용 Operation Attention, Machine Attention 내부에 각각 임베딩됨

- DCAN은 선호도를 단순한 추가 입력으로만 사용하지 않고, 공정과 기계를 서로 바라보는 Attention 가중치 자체를 바꾸도록 설계됨.

❖ Dual Conditional Attention



- Conditional Operation Attention: 이전·현재·다음 공정과 선호도 정보를 결합
- Conditional Machine Attention: 모든 관련 기계, 기계 간 intensity, 선호도 정보를 결합
- 최종 공정·기계 임베딩을 Actor와 Critic에서 사용

III Conditional Operation Attention

❖ Operation Embedding Update

$$h^{O_{ij}}_{ij}{}^{(l+1)} = \sigma(\sum_{p=j-1}^{j+1} \alpha(O_{ij}, O_{ip}) W h^{O_{ip}}_{ip}{}^{(l)} + \alpha(O_{ij}, \lambda_{ij}) W h^{\lambda_{ij}}_{ij}{}^{(l)})$$

주변 공정 메시지와 preference node 메시지를 Attention 가중치에 따라 결합

Symbol	Meaning
$h^{O_{ij}}_{ij}{}^{(1)}$	1번째 층의 공정 O_{ij} 정보 벡터
O_{ip}	같은 Job의 이전·현재·다음 공정
$\alpha(\cdot, \cdot)$	각 정보를 얼마나 참고할지 나타내는 가중치
W	학습 가능한 선형 변환 행렬
λ_{ij}	공정에 연결된 선호도 가상 노드
σ	비선형 활성화 함수

❖ Machine Embedding Update

$$h_k^{M(q+1)} = \sigma(\sum_q \beta(M_k, M_q) Z h_q^{M(q)} + \beta(M_k, \lambda^M) Z h \lambda^{M(q)})$$

다른 기계와 선호도 정보를 결합하여 기계 표현 업데이트

- $\beta(M_k, M_q)$: 기계 M_q 의 정보를 얼마나 참고할지 나타내는 Attention 가중치
- 기계 간 intensity $c(M_k, M_q)$ 는 두 기계가 처리 가능한 공정의 관계를 반영
- 선호도 노드와 기계 사이 intensity는 전체 기계 쌍 intensity의 평균을 사용

❖ Actor

- 최종 공정 임베딩, 기계 임베딩, pair feature를 사용해 가능한 행동의 확률을 출력
- 훈련 중에는 정책 확률에 따라 행동을 샘플링

❖ Critic

- 현재 부분 스케줄의 미래 가치를 평가
- 논문은 목적별 value vector를 제안했으나 단일값 critic 대비 유의미한 향상은 확인되지 않음

❖ Critic

- 현재까지 만들어진 부분 스케줄을 입력받음
 - 현재 상태에서 앞으로 얻을 누적 보상을 예측함
 - 다음 공정-기계 조합을 직접 선택하지는 않음
 - Actor가 선택한 행동이 기대보다 좋았는지 판단하는 기준을 제공함
- 1) 현재 상태 s_t 를 입력받아 가치 $V(s_t)$ 를 예측함
 - 2) Actor가 공정-기계 조합을 하나 선택함
 - 3) 실제 단계별 보상 r_t 가 계산되고 다음 상태 s_{t+1} 가 생성됨
 - 4) Critic이 다음 상태의 가치 $V(s_{t+1})$ 를 다시 예측함
 - 5) 행동 후 결과와 행동 전 기대치를 비교해 Advantage를 계산함
- Critic은 처음에는 랜덤으로 생성.
 - 실제 스케줄 생성 과정에서 얻은 누적 보상과 자신의 예측값을 비교함
 - 예측 오차가 줄어들도록 신경망 파라미터를 반복적으로 업데이트함
 - 학습이 진행될수록 현재 상태와 미래 결과의 관계를 더 정확하게 파악함

❖ Advantage

$$A_t \approx r_t + \gamma V(s_{t+1}) - V(s_t)$$

- $A_t > 0$: Critic이 예상한 것보다 좋은 행동
- $A_t < 0$: Critic이 예상한 것보다 나쁜 행동
- $A_t \approx 0$: 예상과 비슷한 행동
- 절댓값이 클수록 PPO 업데이트에 더 큰 영향을 줌

❖ 실험 목적

- 제안한 WI-DAN과 DCAN이 기존 다목적 최적화 방법보다 좋은 Pareto Set을 생성하는지 확인
- 특히 DCAN의 conditional attention이 단순 preference concatenation 방식인 WI-DAN보다 효과적인지 검증
- 학습된 정책이 FJSP뿐 아니라 다른 잡샵 계열 문제에도 적용 가능한지 확인

❖ 비교 방법

- NSGA-II(2002)
- MOEA/D (2007)
- Hypernetwork(2022)
- WI-DAN
- DCAN
- CP-SAT

❖ 목적 조합

- Makespan + Costs
- Tardiness + Costs
- Makespan + Flowtime + Costs

❖ 추론방식

- Greedy
- Sampling

❖ Results on synthetic instances of the same sizes as the instances used for training

	Size	Metaheuristics		Greedy			Sample			CP-SAT
		NSGA-II	MOEA/D	Hyper	WI-DAN	DCAN	Hyper	WI-DAN	DCAN	
Makespan Costs	10×5	HV	0.7208	0.6616	0.5987	0.6999	0.7104	0.6204	0.7581	0.8313
		Gap	13.30%	20.42%	27.98%	15.81%	14.55%	25.37%	8.81%	0.00%
		Nr. Sol.	8.14	8.81	6.72	2.91	3.46	7.78	7.19	13.20
		Time (s)	247.85	254.85	0.57	0.57	0.87	2.26	2.37	-
	20×5	HV	0.5332	0.4807	0.5083	0.5536	0.5599	0.5268	0.5696	0.6095
		Gap	12.53%	21.14%	16.61%	9.17%	8.14%	13.57%	6.56%	0.00%
		Nr. Sol.	11.96	14.67	6.48	3.30	4.04	9.24	5.72	11.82
		Time (s)	652.04	614.81	1.41	1.47	1.99	8.07	8.51	-
	15×10	HV	0.6809	0.5570	0.7039	0.7650	0.7723	0.7388	0.7920	0.9243
		Gap	26.33%	39.73%	23.85%	17.23%	16.44%	20.07%	14.31%	0.00%
		Nr. Sol.	14.12	16.70	6.95	8.28	9.14	9.17	14.16	18.16
		Time (s)	1694.42	1062.99	2.86	3.07	4.02	21.19	22.72	-
	20×10	HV	0.6289	0.4879	0.7751	0.7966	0.8083	0.7236	0.8073	0.8548
		Gap	26.43%	42.93%	9.32%	6.82%	5.44%	15.36%	5.57%	0.00%
		Nr. Sol.	19.21	19.45	16.43	9.44	13.85	7.43	14.14	14.81
		Time (s)	2798.44	1774.73	4.67	5.05	6.57	38.02	41.66	-
Tardiness Costs	10×5	HV	0.7741	0.7006	0.6335	0.7109	0.7460	0.7103	0.8102	0.9144
		Gap	15.35%	23.38%	30.72%	22.25%	18.42%	22.32%	11.40%	0.00%
		Nr. Sol.	15.41	9.58	6.52	2.66	4.36	11.44	10.28	25.81
		Time (s)	296.60	256.28	0.59	0.59	0.89	2.61	2.66	-
	20×5	HV	0.5331	0.4234	0.5479	0.6372	0.6396	0.5282	0.6645	0.7010
		Gap	23.96%	39.60%	21.84%	9.09%	8.76%	24.64%	5.20%	0.00%
		Nr. Sol.	20.53	17.59	10.23	10.00	10.88	10.87	17.27	11.26
		Time (s)	516.84	619.33	1.55	1.59	2.24	9.47	9.93	-
	15×10	HV	0.6745	0.5131	0.7134	0.7982	0.8094	0.7270	0.8258	0.9505
		Gap	29.04%	46.02%	24.95%	16.03%	14.85%	23.51%	13.12%	0.00%
		Nr. Sol.	22.10	18.96	13.50	15.45	17.04	12.10	26.95	32.39
		Time (s)	1642.20	1048.72	3.12	3.28	4.45	23.90	25.69	-
	20×10	HV	0.5878	0.4229	0.7566	0.8100	0.8112	0.7472	0.8274	0.8498
		Gap	30.83%	50.24%	10.97%	4.68%	4.53%	12.08%	2.63%	0.00%
		Nr. Sol.	24.54	23.23	18.23	17.04	20.03	12.02	29.56	14.54
		Time (s)	2869.48	1745.29	5.51	5.43	7.17	44.66	47.02	-
Makespan Flowtime Costs	10×5	HV	0.6146	0.4969	0.4123	0.4081	0.4647	0.4673	0.4902	0.6831
		Gap	10.02%	27.25%	39.64%	40.26%	31.97%	31.59%	28.24%	0.00%
		Nr. Sol.	167.40	67.01	23.57	7.33	22.64	48.43	39.50	64.98
		Time (s)	229.21	260.88	0.84	0.82	1.14	4.99	4.97	-
	20×5	HV	0.3661	0.2595	0.3800	0.4221	0.4318	0.4021	0.4512	0.5210
		Gap	29.73%	50.20%	27.06%	18.97%	17.11%	22.83%	13.39%	0.00%
		Nr. Sol.	230.89	90.90	31.23	28.15	32.96	71.75	80.70	59.95
		Time (s)	543.45	627.90	2.49	2.52	3.18	18.34	19.39	-
	15×10	HV	0.4654	0.3205	0.4844	0.5583	0.5793	0.4850	0.5782	0.7336
		Gap	36.57%	56.32%	33.97%	23.90%	21.03%	33.89%	21.18%	0.00%
		Nr. Sol.	86.70	37.91	34.10	29.06	43.96	75.54	82.73	68.32
		Time (s)	1649.08	1020.61	4.36	4.52	5.51	35.56	37.33	-
	20×10	HV	0.3834	0.2547	0.5056	0.5822	0.6142	0.5025	0.6038	0.6612
		Gap	42.01%	61.49%	23.54%	11.96%	7.12%	24.00%	8.69%	0.00%
		Nr. Sol.	106.06	41.38	37.29	43.08	57.83	62.65	114.14	60.83
		Time (s)	2797.74	1674.67	7.35	7.72	9.13	64.14	67.44	-

- DCAN은 대부분의 문제 크기와 목적 조합에서 NSGA-II와 MOEA/D보다 높은 Hypervolume을 달성함.
- 가장 큰 20×10문제에서 Sample DCAN은 CP-SAT 대비 Gap 2.25%–4.51%를 기록함.
- DCAN은 WI-DAN보다 대체로 높은 해 품질과 더 많은 비지배해를 생성함.
- Sampling은 Greedy보다 다양한 Pareto 해를 생성하며, 기존 metaheuristic보다 짧은 계산시간을 보임.

❖ JSSP

		Metaheuristics		Greedy			Sample			
Size		NSGA-II	MOEA/D	Hyper	WI-DAN	DCAN	Hyper	WI-DAN	DCAN	CP-SAT
6×6	HV	0.8703	0.8729	0.8354	0.7722	0.8002	0.8450	0.8251	0.8244	0.8778
	Gap	0.85%	0.56%	4.83%	12.02%	8.84%	3.73%	6.00%	6.07%	0.00%
	IGD+	0.0057	0.0027	0.0291	0.0642	0.0486	0.0218	0.0322	0.0342	0.0014
	Nr. Sol.	4.04	4.03	2.73	1.07	1.56	3.08	2.24	2.22	3.17
	Time (s)	998.01	194.50	0.40	0.39	0.61	1.57	1.46	1.82	-
10×10	HV	0.8867	0.8592	0.8767	0.8412	0.8597	0.8844	0.8761	0.8817	0.9203
	Gap	3.66%	6.64%	4.74%	8.60%	6.58%	3.91%	4.81%	4.20%	0.00%
	IGD+	0.0198	0.0421	0.0276	0.0473	0.0371	0.0225	0.0259	0.0242	0.0001
	Nr. Sol.	4.18	4.52	3.48	1.32	2.25	4.23	3.20	4.36	5.35
	Time (s)	980.67	662.93	1.66	1.72	2.35	10.91	11.69	13.49	-
15×15	HV	0.8776	0.7955	0.9003	0.8797	0.8935	0.9133	0.9074	0.9104	0.9494
	Gap	7.57%	16.21%	5.18%	7.34%	5.89%	3.81%	4.42%	4.11%	0.00%
	IGD+	0.0459	0.1107	0.0310	0.0412	0.0347	0.0226	0.0245	0.0242	0.0000
	Nr. Sol.	4.19	2.85	3.97	1.37	2.98	6.34	4.42	5.92	13.27
	Time (s)	4823.55	1694.13	7.57	7.62	9.71	68.58	69.46	82.01	-
20×20	HV	0.8394	0.7791	0.9164	0.9108	0.9145	0.9239	0.9224	0.9245	0.9576
	Gap	12.34%	18.65%	4.31%	4.89%	4.50%	3.52%	3.68%	3.46%	0.00%
	IGD+	0.0832	0.1327	0.0263	0.0291	0.0277	0.0217	0.0216	0.0215	0.0000
	Nr. Sol.	3.02	2.50	4.33	3.21	4.18	6.60	5.68	6.85	10.50
	Time (s)	19926.21	4561.18	28.24	28.29	30.35	267.84	275.62	278.54	-

- 작은 문제에서는 일부 metaheuristic이 DCAN보다 우수한 경우가 존재함.
- 문제 크기가 커질수록 DCAN의 성능이 향상되며, metaheuristic보다 짧은 실행시간을 유지함.
- CP-SAT을 능가하지는 못했으나, 문제 규모가 커질수록 성능 차이가 감소함.

❖ FFSP

		Metaheuristics		Greedy			Sample			
Size		NSGA-II	MOEA/D	Hyper	WI-DAN	DCAN	Hyper	WI-DAN	DCAN	CP-SAT
15×5	HV	0.4492	0.3202	0.3808	0.3898	0.4014	0.4175	0.4336	0.4385	0.5541
	Gap	18.92%	42.20%	31.27%	29.66%	27.56%	24.65%	21.74%	20.85%	0.00%
	IGD+	0.0892	0.1878	0.1220	0.1185	0.1034	0.0986	0.0903	0.0814	0.0013
	Nr. Sol.	417.96	163.73	30.27	30.94	33.38	77.34	104.36	108.80	76.52
	Time (s)	639.38	359.20	1.75	1.73	2.47	13.26	13.14	15.05	-
20×4	HV	0.4039	0.2925	0.4057	0.3082	0.4200	0.4374	0.3647	0.4521	0.5424
	Gap	25.54%	46.08%	25.20%	43.17%	22.56%	19.35%	32.76%	16.65%	0.00%
	IGD+	0.0970	0.1918	0.0553	0.0837	0.0495	0.0420	0.0557	0.0355	0.0098
	Nr. Sol.	477.05	207.32	46.69	13.02	50.03	151.83	111.49	177.35	75.71
	Time (s)	699.80	389.60	2.13	2.20	2.65	16.84	16.92	17.94	-

- DCAN은 Hypervolume 기준으로 metaheuristic과 경쟁하거나 더 높은 성능을 보임.
- CP-SAT보다 해 품질은 낮지만, 훨씬 짧은 시간에 Pareto Set을 생성함.

❖ Conclusion

- 제안한 DCAN은 대부분의 synthetic FJSP 실험에서 NSGA-II와 MOEA/D보다 높은 Hypervolume을 달성하면서도 훨씬 짧은 계산시간을 보임.
- 특히 문제 규모가 커질수록 기존 탐색 기반 방법의 성능과 속도가 저하되는 반면, DCAN은 학습된 정책을 활용해 비교적 안정적으로 고품질 Pareto Set을 생성함.
- 또한 DCAN은 preference vector를 단순히 입력에 연결한 WI-DAN보다 대체로 더 높은 해 품질과 더 많은 비지배해를 생성함.
- 이는 목적 선호도를 Attention 내부에 직접 반영하는 구조가 서로 다른 trade-off를 구분하고, 다양한 Pareto solution을 생성하는 데 효과적임을 보여줌.
- 따라서 DCAN은 하나의 최적해만 찾는 모델이 아니라, 목적 선호도에 따라 빠른 스케줄, 저비용 스케줄, 균형형 스케줄 등 다양한 대안을 제공하는 다목적 스케줄링 모델임.

❖ Novelty

- 선호도 벡터를 단순히 입력에 결합하지 않고, Operation-Machine Attention 내부의 가상 노드로 반영한 DCAN을 제안함.
- 목적별 하한의 변화를 이용한 stepwise reward로 긴 스케줄링 과정의 delayed reward 문제를 완화함.
- 선호도별 모델을 따로 학습하지 않고, 하나의 정책으로 다양한 Pareto 스케줄을 생성함.